

Praetor v2

The governed runtime for agentic GRC.

Run AI agents to do your compliance work. Govern the AI you ship. One control plane.

Part 0: How to read this document

This is a combined business plan and codebase plan for a hackathon build, restructured around a unified thesis: Praetor is one control plane that exposes two surfaces, both running on the same primitives.

- **Workflow Mode** runs agentic GRC work on behalf of the customer. Ingest frameworks, legislation, internal policies, code, business processes. Hook into the customer's stack via MCPs and connectors. Agents reason over the corpus, find issues, propose fixes, and route them through approval and application.
- **Supervision Mode** instruments the customer's own production AI systems and agents. Runtime policy enforcement, memory governance, sandboxed remediation, audit-ready evidence.

Both modes share the same data model, sandbox runtime, hook framework, policy engine, evidence pipeline, and audit output. The novel claim of this product is that those two surfaces *should* share a control plane, because the agents that automate compliance work are themselves regulated artefacts.

Working name: **Praetor**. Headline: *The governed runtime for agentic GRC*. Title slide subtitle: *Run AI agents to do your compliance work. Govern the AI you ship. One control plane.*

Part 1: Business Plan

1.1 Thesis in one paragraph

Compliance, risk, and governance teams are drowning in repetitive analytical work (mapping new regulations to existing controls, scanning code and business processes for policy gaps, assessing vendors, reconciling overlapping frameworks) while simultaneously being asked to govern a new generation of production AI systems and agents that traditional GRC tooling was never designed to watch. The two problems look distinct but share a substrate: both reduce to "agents reasoning over a corpus of obligations, observing a body of state, and producing evidence and corrective action." Praetor builds that substrate once and exposes it as two products. The agents we run on the customer's behalf to automate GRC work are themselves governed by the runtime control plane we offer for the customer's production AI. That dogfooding is not a marketing line; it is the architectural answer to the trust question every buyer of an AI-driven GRC platform asks within the first ten minutes.

1.2 Why now

Three forces converge in the 2026 to 2028 window.

First, regulatory surface area is expanding and becoming concrete. The EU AI Act is in active 2026 to 2028 implementation with simplification amendments in trilogue. DORA is directly applicable. NIS2 is being transposed across member states. ISO 42001 is appearing in enterprise procurement. NIST AI RMF and the AI Agent Standards Initiative are converging on runtime governance language. Every one of these creates analytical work that today is done by humans reading PDFs.

Second, agent incidents are now boardroom incidents. The OpenClaw exposure wave, the Moltbook token leak, and the OWASP agent threat catalogue have moved agent security from research into procurement. Boards now ask "how do we know our agents are not the next headline."

Third, the workflow automation category is hitting its ceiling for compliance work. Zapier, n8n, Make, and Workato can wire together SaaS APIs with point-in-step LLM calls, but they cannot reason over a regulatory corpus, cannot cite obligations in their outputs, cannot sandbox or replay their own steps, and cannot govern the LLM steps they execute. That ceiling is exactly where agentic GRC begins.

1.3 Differentiation: how Praetor is not a Zapier or an n8n

This deserves to be said cleanly because judges and buyers will both ask.

Axis	Zapier / n8n / Make	Praetor
Step model	Deterministic call to a SaaS API; AI is one step among many	Sandboxed agent run with reasoning, tool access, and memory
Data primitives	Generic records and fields	Obligations, controls, findings, evidence, audit packets
Corpus awareness	None	Frameworks, legislation, internal policies as queryable, citable corpora
Execution isolation	Multi-tenant cloud, no per-step sandbox	Per-step Docker (optionally gVisor) sandbox with replayable inputs
Tamper-evidence	Logs	Hash-chained event log per asset and per workflow run
Connector surface	Vendor-curated, fixed	MCP-native; any MCP server is a hook; native connectors as well
Output	Side effect (post, write, send)	Structured Finding and ProposedChange with citations and sandbox results
Self-governance	None	Workflow agents governed by the same runtime control plane the customer buys
Audit artefact	None native	Signed audit packet with obligation chain

The two-line version: Zapier is task plumbing for SaaS APIs. Praetor is governed agentic work for compliance, where every step is sandboxed, every output cites an obligation, and the agents doing the work are themselves under runtime governance.

1.4 Differentiation against the GRC and AI governance categories

Capability	Trust platforms (Vanta/Drata)	AI gov platforms (Credo/Holistic)	Cloud security (Wiz/Snyk)	Workflow tools (Zapier/n8n)	Praetor
Continuous evidence collection	Yes	Some	No	No	Yes
Multi-framework obligation mapping	Yes	Yes	No	No	Yes
Agentic GRC work execution	No	No	No	Partial	Yes
AI/agent runtime telemetry	No	Partial	Partial	No	Yes
Memory governance	No	No	No	No	Yes
Sandboxed step execution	No	No	Partial	No	Yes
MCP-native hook surface	No	No	No	Emerging	Yes
Self-governing workflow agents	No	No	No	No	Yes
Auto-generated audit packets	Yes	Yes	No	No	Yes

The defensible moat is the bottom four rows in combination, anchored by self-governance. No single competitor is positioned to cover that surface in one product.

1.5 What we are not

Three explicit anti-positions, said out loud on a slide if needed.

- We are not replacing lawyers or auditors. Our agents produce confidence-scored mappings and proposals; ambiguous cases route to humans.
- We are not a generic GRC suite. We do not chase SOC 2 evidence as a primary wedge. Existing trust platforms are good at that and we integrate with them.
- We are not a generic workflow automation tool. Our workflows are domain-typed, sandboxed, and governed by design, which is not what a generic workflow tool wants to be.

1.6 Target buyer and ICP

Primary ICP for the post-hackathon company:

- Mid-market to enterprise, 200 to 5,000 employees.
- AI is in production in customer-facing or revenue-affecting workflows, or AI is a stated near-term roadmap item.
- At least one of: ISO 42001 track, EU exposure, regulated sector (financial services, healthcare, public sector suppliers, critical infrastructure).
- GRC team is small relative to the scope of obligations they own (the repetitive-work pain is real and budgeted).
- Existing trust platform already in place; we integrate, not displace.

Buyer titles: Head of AI Governance, CISO, Chief Compliance Officer, VP Platform Engineering, Head of GRC. Champion is usually a senior security engineer, an AI platform lead, or a GRC analyst who is paged when the regulator emails or when an agent does something it should not have.

1.7 Revenue model (post-hackathon)

- **Platform fee** by tier (Starter, Growth, Enterprise) based on governed AI assets, number of active workflows, and corpus volume.
- **Usage fee** per sandbox run beyond a generous bundled allowance (covers both workflow steps and remediation tests).
- **Compliance content subscription** for jurisdiction and sector packs (EU AI Act, ISO 42001, NIST AI RMF, GDPR, DORA, MITRE ATLAS, OWASP, sector packs for financial services, healthcare, public sector). This is the recurring high-margin layer.
- **Workflow template marketplace** for community and partner-contributed templates, with a revenue share.

For the hackathon itself, no monetisation shown. We discuss the model only if asked.

1.8 Demo narrative (the unified story)

A fictional company, **Northwind Health**, runs a customer-support agent that has access to a patient knowledge base, an outbound email tool, and a refund tool. They also have a small GRC team that has just been told to demonstrate ISO 42001 readiness within a quarter, while the EU AI Act high-risk obligations for health-eligibility AI are coming into force.

The demo runs both surfaces in one flow.

1. The GRC analyst opens Praetor and creates a Workflow Run from the **Code Compliance Scan** template. They hook GitHub via MCP and point at the support-bot repo. They select the corpora to reason over: the company's internal data minimisation policy, ISO 42001 (clauses 7 to 10), GDPR (Article 5), and OWASP Agent Top 10.
2. The workflow agent kicks off. Each step runs in a sandbox. The audience watches its thinking, retrievals, and tool calls stream into the UI. At the same time, the supervision panel shows that the workflow agent is itself a governed asset, with its own policy decisions in the right pane.
3. The workflow agent produces a **Finding**: the support-bot's `send_email` tool does not validate recipient domains, which violates the internal data minimisation policy, which traces to ISO 42001 8.3 and GDPR 5(1)(c). The finding includes citations and a confidence score.
4. The agent attaches a **ProposedChange**: a code patch tightening the recipient validator and a config change narrowing the tool's allowlist. It runs the patch in the sandbox against a battery of replayed inputs (including known prompt injection patterns) and reports green.
5. Praetor opens an approval. The analyst approves in-product. The change is applied as a GitHub pull request via the outbound MCP hook.
6. We switch into Supervision Mode. The original support-bot is now running with the patch. We replay the malicious-email injection prompt. The agent's `send_email` tool refuses internally. The policy engine logs the refusal as evidence rather than as a violation, because the patch caught it.
7. The analyst clicks **Generate Audit Packet**. A signed PDF appears covering the workflow execution, the proposed change, the approval, the applied patch, and the supervision evidence afterwards. The obligation chain is rendered as a graph.

The wow moment is the dual view: workflow agents and production agents on the same screen, the same hash chain, the same obligation graph, the same audit output. Both modes are visibly the same product.

1.9 Why this wins a hackathon

It demonstrates a live, working product, not a deck. It shows multiple agents (workflow agent, production agent, remediation agent, audit agent) running in coordinated flow, which photographs well. It connects to a real and timely regulatory story without being a deck about regulation. It includes a clear technical wedge (sandboxed agentic steps + corpus reasoning + self-governance + MCP hooks) that judges can evaluate independently of business framing. It produces a tangible artefact (the audit packet PDF) that the audience walks away remembering. And the differentiation against Zapier is sharp enough to answer the most predictable judge question without breaking stride.

Part 2: Technical Architecture

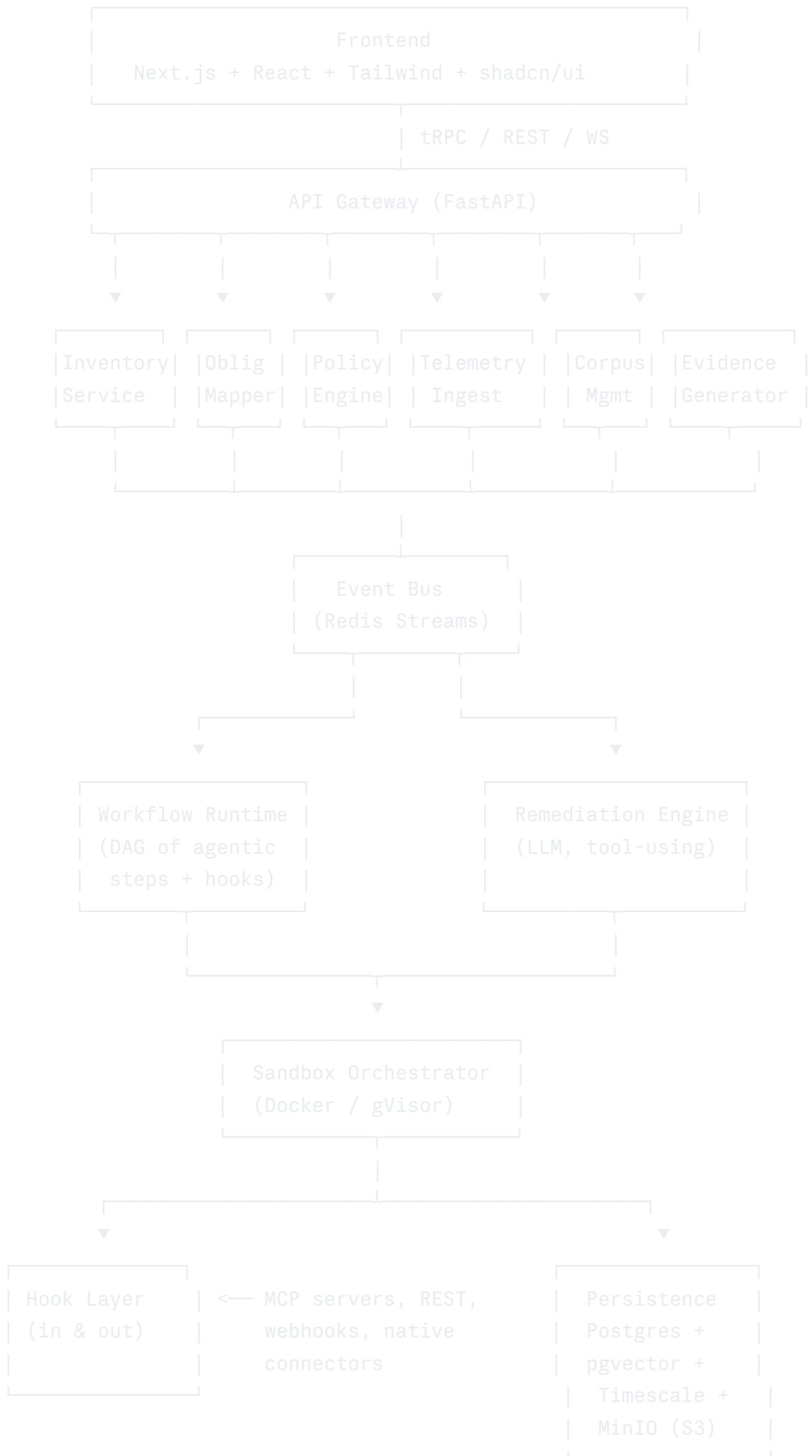
2.1 Architectural principles

Six principles drive every design decision.

1. **Everything is an event.** Prompts, tool calls, memory writes, hook calls, workflow step starts and ends, approvals, refusals, configuration changes are first-class events with provenance and a hash chain.
2. **Policy is code, controls are tests, evidence is an output.** Controls are deterministic functions over the event stream, parameterised by obligations.
3. **Enforcement on the hot path. Reasoning on the warm path.** Latency-sensitive checks (tool permission, memory write quarantine, workflow step gate) are deterministic and inline. LLM-driven obligation mapping, finding generation, and remediation reasoning run asynchronously.
4. **Sandboxes are the only place mutation happens before approval.** No proposed change runs against production until it has run green in an isolated sandbox and a human (or an explicit policy) has signed off.
5. **The audit trail is the source of truth.** Every action against an asset or in a workflow is reconstructable from the event log alone.
6. **Workflow agents are governed assets.** Anything the platform runs on the customer's behalf is itself an asset under supervision, with the same hash chain and the same audit treatment.

2.2 System overview

Praetor has eight major subsystems plus a frontend. The new subsystem (Workflow Runtime) and the new shared layer (Hooks and Corpus) are highlighted.



2.3 Tech stack

Choices optimised for hackathon velocity, not for production hardening.

Layer	Choice	Why
Frontend	Next.js 15 (App Router), React 19, Tailwind, shadcn/ui, Framer Motion, Recharts, React Flow	Fast, modern, looks good. React Flow for the workflow graph and obligation graph.
API gateway	FastAPI (Python 3.12) + Pydantic v2	Fast iteration, strong types, AI ecosystem fit.
Realtime	WebSockets via FastAPI + Redis pub/sub	Streams agent traces, workflow progress, policy decisions to the UI live.
Event bus	Redis Streams	One container, durable enough for a demo, simple consumer groups.
Database	PostgreSQL 16 with pgvector and TimescaleDB	Single store for relational, vector, and time-series.
Object storage	MinIO (S3-compatible)	Local, free, real S3 API for evidence, corpora, sandbox artefacts.
Policy engine	Open Policy Agent (OPA) on the hot path; LLM reasoner on the warm path	OPA is the standard for policy-as-code.
Sandbox runtime	Docker; gVisor (<code>runsc</code>) when available	Ships on a laptop. Used by both Workflow Runtime and Remediation Engine.
Workflow engine	Custom thin DAG runner over the sandbox primitive	Off-the-shelf engines (Temporal, Prefect) are too heavy and not domain-typed.
Agent orchestration	LangGraph for workflow agent steps and remediation engine; raw Anthropic SDK for demo agent	LangGraph gives inspectable graphs; raw SDK keeps demo agent transparent.
Hook framework	MCP SDK (Anthropic) + a thin generic HTTP/webhook bridge	MCP is the emerging standard; we ride it.
Corpus indexing	pgvector for embeddings; <code>tantivy-py</code> or <code>ts_query</code> for keyword	Hybrid retrieval over ingested regs and policies.
LLM provider	Anthropic Claude (Sonnet for cheap, Opus for high-stakes reasoning)	Best tool use and structured output today.
Auth (demo)	Single dev user via <code>Authorization: Bearer dev</code>	Auth is not the demo.

2.4 Subsystem responsibilities

2.4.1 Inventory service

Maintains the catalogue of governed entities. Unchanged from v1 in shape, expanded in scope.

- **AI Systems** and **Agents** owned by the customer (supervision targets).
- **Tools, Memory Stores, Datasets, Models.**
- **Operators** (the legal entity playing AI Act provider, deployer, distributor, or importer roles).
- **Workflow Agents** are also Assets, with `type = "workflow_agent"` and `owner = "praetor:platform"`. This is what makes self-governance literal: the same UI, same policy engine, same audit treatment as any customer-owned agent.

2.4.2 Obligation mapper

Turns inventory plus operator role plus corpus into the set of applicable obligations. v1 logic preserved. New: the mapper now takes ingested corpora as input alongside curated framework content, so internal policies and customer-specific obligations are first-class citizens. The same confidence scoring and human-review escalation apply.

2.4.3 Policy engine

Two-layer design unchanged. Hot path is OPA on Rego, sub-10ms. Warm path is LLM reasoning over event context, obligation graph, and similar past decisions retrieved by vector search. Both surfaces (workflow steps and supervised agent events) flow through the same engine.

2.4.4 Telemetry ingest

High-throughput ingest endpoint. Sources:

1. **SDK** (Python and TypeScript) wrapping Anthropic, OpenAI, and LangGraph runtimes.
2. **Sidecar** proxy for closed-source agents.
3. **Workflow Runtime** itself, which emits events for every workflow step, hook call, and sandbox boundary.

Events are normalised to the `AgentEvent` schema and placed on Redis Streams. Consumers include the policy engine, memory inspector, evidence generator, and the frontend live view.

2.4.5 Sandbox orchestrator

Spins isolated execution environments on demand. Used by:

- 1. **Workflow Runtime** for every agent step.
- 2. **Remediation Engine** for testing proposed changes.
- 3. **Counterfactual replay** (post-hackathon).

Each sandbox is a Docker container (gVisor when available) with no external network except a controlled `praetor-mocks` bridge, read-only root with a writable overlay at `/sandbox/work`, capped resources, and a 300s wall clock. Telemetry from inside the sandbox flows back tagged with `sandbox_id` and `workflow_run_id` (when applicable).

2.4.6 Workflow Runtime (new)

The new headline subsystem. Owns three things: workflow definitions, workflow runs, and the step execution loop.

A **Workflow** is a DAG of typed steps with declared inputs, outputs, hooks, and policies. Step types:

Type	Description
<code>agent</code>	Runs an LLM agent in a sandbox with declared tools, memory, and corpus access. Produces structured output.
<code>corpus.query</code>	Retrieval against a named corpus; returns citations.
<code>hook.in</code>	Pulls data from an external system via a registered hook (MCP or native connector).
<code>hook.out</code>	Pushes data to an external system via a registered hook (e.g. open a Jira ticket, post a GitHub PR).
<code>transform</code>	Pure deterministic transformation over previous step outputs (shape, filter, merge).
<code>gate.policy</code>	Evaluates a policy; halts or branches based on outcome.
<code>gate.human</code>	Routes for human review or approval; blocks until completed.
<code>finding.emit</code>	Emits a structured Finding into the platform.
<code>change.propose</code>	Emits a structured ProposedChange tied to a Finding.

Every `agent` step runs in its own sandbox. The Workflow Runtime is what makes the differentiation against Zapier real: a step is not a SaaS API call, it is a sandboxed agent with corpus access, governed by the same policy engine that watches the customer's production agents.

A **WorkflowRun** is one execution of a Workflow. It has its own hash-chained event log, its own set of step runs, and its own audit trail. It is itself an `Asset` with `type = "workflow_run"` so it can be queried, monitored, and audited like any other asset.

Triggers: manual (UI), schedule (cron), webhook (incoming HTTP), and event-based (a violation in supervision mode triggers a remediation workflow).

Templates shipped at hackathon scope:

1. **Code Compliance Scan:** scan a repo against MITRE ATLAS, OWASP, internal coding policy, named regulations.
2. **Process Compliance Scan:** scan a BPM doc, data flow diagram, or service description against data minimisation, GDPR, sector-specific obligations.
3. **Vendor Risk Assessment:** ingest a vendor's SOC 2 or ISO certification, map gaps against the customer's obligation set.
4. **Policy Gap Analysis:** ingest a new regulation or internal policy, find gaps in existing controls.
5. **Continuous Control Monitoring:** scheduled run of all enabled controls against current state, produce evidence.

A workflow definition example (full schema in Part 3):

yaml

```
name: code_compliance_scan
trigger: manual
inputs:
  repo_url: { type: string, required: true }
  corpus_ids: { type: list[string], required: true }
steps:
- id: pull
  type: hook.in
  hook: github_mcp
  args: { url: "{{ inputs.repo_url }}", paths: ["**/*.py", "**/*.ts"] }

- id: scan
  type: agent
  sandbox: { mem_mb: 2048, wall_s: 240 }
  agent:
    model: claude-sonnet-4
    system_prompt_ref: prompts/code_compliance_scanner.md
    tools: [grep, ast_parse, embed_search]
    corpora: "{{ inputs.corpus_ids }}"
  inputs: { files: "{{ steps.pull.files }}" }
  outputs: { findings: list[Finding] }

- id: gate
  type: gate.policy
  policy: praetor.controls.workflow_findings_gate
  inputs: { findings: "{{ steps.scan.findings }}" }

- id: emit
  type: finding.emit
  inputs: { findings: "{{ steps.scan.findings }}" }

- id: propose
  type: agent
  sandbox: { mem_mb: 2048, wall_s: 300 }
  agent:
    model: claude-opus-4
    system_prompt_ref: prompts/remediation_proposer.md
    tools: [read_repo, write_diff, run_tests]
  inputs: { findings: "{{ steps.scan.findings }}", repo: "{{ steps.pull.files }}" }
  outputs: { proposed_changes: list[ProposedChange] }

- id: approve
  type: gate.human
  routes_to: { role: "compliance_lead" }
  inputs: { changes: "{{ steps.propose.proposed_changes }}" }
```

```

inputs: { changes: "{{ steps.propose.proposed_changes }}" }

- id: open_pr
  type: hook.out
  hook: github_mcp
  when: "{{ steps.approve.outcome == 'approved' }}"
  args: { repo: "{{ inputs.repo_url }}", changes: "{{ steps.propose.proposed_changes }}" }

```

2.4.7 Hook Layer (new shared subsystem)

Hooks are the unified abstraction for everything that crosses the platform boundary. Three hook directions:

- **Inbound:** pull data into a workflow step or into a corpus. Examples: GitHub repo files, Confluence pages, S3 objects, Jira issues, a vendor's SOC 2 report PDF, a customer's BPM XML, a supervised agent's telemetry.
- **Outbound:** push data out. Examples: open a GitHub pull request, post a Slack message, create a Jira ticket, write to ServiceNow, file a finding into an existing GRC tool.
- **Bidirectional:** long-lived integrations such as supervising a running agent (telemetry in, policy decisions out).

Hook implementations:

1. **MCP servers** (the primary path). The platform speaks MCP as a client to consume any MCP server's tools and resources. We ship a small registry of pre-configured MCP integrations (GitHub, Linear, Jira, Slack, Notion, Confluence, Drive) and let users add more.
2. **Native connectors** for high-value integrations where MCP coverage is thin or where we want richer semantics (e.g. our own SDK and sidecar for supervision).
3. **Generic HTTP / webhook** for everything else.

The platform is also itself an MCP server for downstream consumers, exposing read-only resources (assets, obligations, controls, evidence) and a small set of safe tools (open audit packet, list findings). This lets external agents query Praetor's state without going through the UI.

Hook governance: every hook call is an event on the chain. Inbound hooks are tagged with the corpus or step they fed; outbound hooks require a `gate.human` or an explicit policy that whitelists their effect class. This means an outbound `hook.out` to GitHub PR cannot execute without approval unless the workflow is marked low-risk and the policy permits it.

2.4.8 Corpus Management (new shared subsystem)

A **Corpus** is a versioned, attributable, queryable body of documents, indexed for hybrid retrieval (vector + keyword + structural).

Document types at hackathon scope:

- **Regulation** (EU AI Act, GDPR, DORA, NIS2, sector-specific). Structured citations are first-class; ingestion produces both a Document and hydrates Obligation records.
- **Standard** (ISO 42001, NIST AI RMF, SOC 2, OWASP). Same as Regulation but distinguished for filtering.
- **Internal policy** (data minimisation policy, AI use policy, secure coding standard). Ingested by the customer; treated identically to Regulations for the purposes of citation in Findings.
- **Code repository** (a snapshot of a repo). Indexed for semantic and AST-based retrieval.
- **Process artefact** (BPMN XML, data flow diagram, service description, runbook). Parsed where structure is known, indexed as text otherwise.
- **Evidence reference** (a vendor SOC 2 report, a third-party penetration test). Ingested for use in vendor risk workflows.

Each Document has provenance (where it came from, when, by whom), version history, and a content hash. Corpora are versioned snapshots so that a workflow run can be reproduced months later against the exact corpus state used at the time.

Retrieval is hybrid: dense embeddings via pgvector for semantic similarity, plus a keyword index, plus structured filters (framework, citation, jurisdiction, sector). Workflow steps that retrieve from a corpus emit `corpus.query` events with the query, the returned chunks, and the relevance scores; this is part of the audit trail.

2.4.9 Remediation Engine

Unchanged from v1 in spirit, but now subsumes a more general pattern. Remediation is a specific case of `change.propose`. A violation in supervision mode triggers a remediation workflow (selected from the template library), which uses the same Workflow Runtime, the same sandboxes, and the same approval mechanism as any other workflow.

This collapses two subsystems into one and makes the architecture more honest: there is one way the platform proposes and tests changes, and remediation is just a workflow whose trigger is a violation.

2.4.10 Evidence Generator

Subscribes to events and policy decisions and continuously assembles `EvidenceRecord`s linked to controls. On demand, produces an `AuditPacket` covering both surfaces. The packet now spans:

- Asset state (supervised assets and workflow agents).
- Workflow runs and step-by-step events.
- Findings and proposed changes, with sandbox results and approver chains.
- Supervision policy decisions and violations.
- Hook calls (inbound and outbound), with redactions.
- Obligation chain diagrams.

PDF generation continues server-side via ReportLab, with a JSON sidecar for machine verification.

2.5 Frontend architecture

Eight surface-level views, all built on shadcn/ui.

1. **Dashboard.** Asset count by tier, open findings, in-flight workflows, supervision violations, obligation coverage, recent audit packets.
2. **Inventory.** Assets searchable, filterable by type. Workflow Agents and Production Agents in the same list, distinguished by a chip.
3. **Asset Detail.** Three-pane live view (thoughts, memory, policy decisions). Used for both workflow agent runs and production agent supervision; the layout is identical and the experience reinforces "same product, two surfaces."
4. **Workflows.** Library of templates plus saved workflows. Click a template to instantiate, configure inputs and hooks, run.
5. **Workflow Run.** A live DAG view with step states, expandable per-step traces, and the Asset Detail panel for any running agent step. Findings and proposed changes accumulate in a side panel as steps complete.
6. **Corpora.** Manage ingested documents and corpus versions. Search across corpora. See which obligations were hydrated from which Regulation.
7. **Hooks.** Registered MCP servers and native connectors, with health, scope, and recent calls.
8. **Obligations.** Graph view of obligations to controls to assets, plus filter for "needs human review."
9. **Sandboxes.** Active and historical runs, with input snapshot, applied diff, replay results.
10. **Evidence.** Audit packet builder; pick a period, an asset (or all), an obligation set, render a PDF.

A global alerts tray shows live findings, supervision violations, and pending approvals.

2.6 Deployment topology

For the hackathon: `(docker compose up)` brings up Postgres (with pgvector + Timescale), Redis, MinIO, OPA, the FastAPI gateway, the worker pool (Celery or RQ), the Workflow Runtime worker, the Sandbox Orchestrator, the Next.js frontend, the demo Northwind agent, and three pre-configured MCP servers (a stub GitHub MCP, a stub Slack MCP, and a generic file-based MCP for local docs).

For production: Kubernetes, sandbox orchestrator on Kata or Firecracker, separate managed Postgres, separate vector store, MCP servers as managed sidecars or external SaaS endpoints.

Part 3: Data Model and API Surface

3.1 Core entities

All entities have `(id)`, `(urn)`, `(created_at)`, `(updated_at)`, `(created_by)`, `(version)`. URNs follow `urn:praetor:<entity-type>:<tenant>:<slug>`.

Asset

```
Asset {
  id, urn,
  type ("ai_system" | "agent" | "tool" | "memory_store" | "dataset" |
        "model" | "workflow_agent" | "workflow_run"),
  name, description, owner_id, risk_tier (L1..L4),
  lifecycle (discovered|classified|governed|deprecated),
  metadata (jsonb), tags[],
  parent_asset_id?,
  jurisdictions[], data_classifications[], sectors[],
  config (jsonb), fingerprint (sha256)
}
```

`(workflow_agent)` and `(workflow_run)` types are what bring workflows into the supervision umbrella. They carry the same telemetry, the same hash chain, and the same policy treatment as customer-owned agents.

OperatorRole, Obligation, Control, AssetObligationLink

Unchanged from v1.

AgentEvent (time-series, hot table)

```
AgentEvent {
  id, ts, asset_id, run_id, parent_event_id?,
  workflow_run_id?, workflow_step_id?,
  type, actor, payload (jsonb), payload_redacted (jsonb),
  hash_chain_prev, hash_chain_self
}
```

Two new optional fields tie events to workflow runs and steps for unified querying.

PolicyDecision

Unchanged.

Workflow (new)

```
Workflow {
  id, urn, name, description,
  definition (yaml/json),           // the DAG spec
  trigger ("manual" | "schedule" | "webhook" | "event"),
  trigger_config (jsonb),
  inputs_schema (jsonb), outputs_schema (jsonb),
  required_hooks[], required_corpora[],
  default_policy_set,               // policies bound to step types
  template_origin?                  // if instantiated from a template
}
```

WorkflowRun (new)

```
WorkflowRun {
  id, workflow_id, asset_id (always type=workflow_run),
  triggered_by, triggered_at, finished_at?,
  status ("queued" | "running" | "succeeded" | "failed" | "cancelled" |
          "awaiting_approval"),
  inputs (jsonb), outputs (jsonb),
  step_runs[] (foreign refs),
  evidence_record_ids[]
}
```

StepRun (new)

```
StepRun {
  id, workflow_run_id, step_id, step_type,
  status, started_at, finished_at?,
  inputs_redacted (jsonb), outputs_redacted (jsonb),
  sandbox_run_id?,                // for agent steps
  hook_call_id?,                  // for hook steps
  policy_decision_id?,            // for gate.policy steps
  approval_id?,                  // for gate.human steps
  emitted_finding_ids[], emitted_proposal_ids[]
}
```

Hook (new)

```
Hook {
  id, urn, name, kind ("mcp" | "native" | "http"),
  direction ("in" | "out" | "both"),
  endpoint, auth_ref,              // secret reference
  scopes[],                       // what tools/resources it exposes
  effect_radius ("internal" | "external_trusted" | "external_public"),
  enabled, last_health_check, version
}
```

HookCall (new)

```
HookCall {
  id, ts, hook_id, direction, workflow_run_id?, step_run_id?,
  inputs_redacted (jsonb), outputs_redacted (jsonb),
  status, latency_ms, errors[],
  policy_decision_id?
}
```

Corpus (new)

```
Corpus {
  id, urn, name, description,
  kind ("regulation" | "standard" | "internal_policy" | "code_repo" |
        "process_artefact" | "evidence_reference"),
  version, parent_corpus_id?,      // version chain
  document_count, indexed_at, retention
}
```

Document (new)

```

Document {
  id, corpus_id, source_uri, content_hash,
  title, citation, framework?, jurisdiction?, sector?,
  text_path (S3), // raw text
  chunks[] (foreign refs), // for retrieval
  parsed_structure (jsonb)? // when AST or BPMN parsing applied
}

DocumentChunk {
  id, document_id, ord,
  text, embedding (vector), keyword_tokens,
  citation_path // e.g. "Article 5 / paragraph 1 / point
c"
}

```

Finding (new)

```

Finding {
  id, urn, workflow_run_id?, asset_id,
  title, description,
  severity ("critical" | "high" | "medium" | "low" | "info"),
  obligations_cited[], // urns
  documents_cited[], // urn + chunk ord pairs
  confidence (0..1),
  status ("open" | "accepted" | "rejected" | "remediated" | "wontfix"),
  reviewer?, reviewed_at?,
  proposed_change_ids[]
}

```

ProposedChange (new; supersedes RemediationProposal)

```
ProposedChange {
  id, urn, finding_id,
  kind ("config" | "code" | "policy" | "process" | "doc"),
  diff (text), diff_format ("unified" | "json-patch" | "config" | "markdown"),
  target_asset_id?, target_hook_id?,
  obligations_addressed[],
  residual_risk_estimate (0..1),
  sandbox_run_id?,
  status ("draft" | "tested" | "awaiting_approval" | "approved" |
    "rejected" | "applied"),
  approver?, approved_at?, applied_at?,
  apply_via_hook_id? // e.g. github_mcp for opening a PR
}
```

SandboxRun, EvidenceRecord, AuditPacket

SandboxRun adds optional `workflow_run_id` and `step_run_id` for traceability.

EvidenceRecord and AuditPacket otherwise unchanged in shape but now span both surfaces.

3.2 AgentEvent types (extended)

The v1 set, plus:

Type	Emitter	Payload key fields
<code>workflow.run.started</code>	Workflow Runtime	<code>workflow_id</code> , <code>inputs_redacted</code>
<code>workflow.run.finished</code>	Workflow Runtime	<code>status</code> , <code>outputs_redacted</code>
<code>workflow.step.started</code>	Workflow Runtime	<code>step_id</code> , <code>step_type</code>
<code>workflow.step.finished</code>	Workflow Runtime	<code>step_id</code> , <code>status</code> , <code>outputs_redacted</code>
<code>hook.in.called</code>	Hook Layer	<code>hook_id</code> , <code>inputs_redacted</code> , <code>outputs_summary</code>
<code>hook.out.called</code>	Hook Layer	<code>hook_id</code> , <code>inputs_redacted</code> , <code>outputs_summary</code>
<code>corpus.query</code>	Workflow Runtime	<code>corpus_id</code> , <code>query</code> , <code>chunks_returned[]</code> , <code>top_score</code>
<code>finding.emitted</code>	Workflow Runtime	<code>finding_id</code> , <code>severity</code> , <code>obligations_cited[]</code>
<code>change.proposed</code>	Workflow Runtime	<code>proposed_change_id</code> , <code>kind</code> , <code>target</code>
<code>change.applied</code>	Workflow Runtime	<code>proposed_change_id</code> , <code>apply_via_hook_id</code> , <code>result</code>

The hash chain still applies per asset; for a workflow run, the asset is the `workflow_run` asset.

3.3 API surface (extensions to v1)

Workflows:

- `GET /workflows` (list templates and saved workflows)
- `POST /workflows` (create or save)
- `GET /workflows/{id}`
- `POST /workflows/{id}:run` (instantiate with inputs)
- `GET /workflow-runs/{id}` (status + step runs + emitted artefacts)
- `POST /workflow-runs/{id}:cancel`
- `WS /ws/v1/workflow-runs/{id}/stream` (live progress)

Hooks:

- `GET /hooks`
- `POST /hooks` (register an MCP server, native connector, or webhook)
- `POST /hooks/{id}:test`
- `GET /hook-calls?workflow_run_id=...`

Corpora:

- `GET /corpora`
- `POST /corpora` (create)
- `POST /corpora/{id}/documents:ingest` (file upload, URL pull, or hook reference)
- `GET /corpora/{id}/documents`
- `POST /corpora/{id}:search` (hybrid retrieval)
- `POST /corpora/{id}:snapshot` (freeze a version)

Findings and ProposedChanges:

- `GET /findings` (filter by workflow_run, asset, status, severity, framework)
- `GET /findings/{id}`
- `POST /findings/{id}:accept` / `:reject`
- `POST /proposed-changes/{id}:sandbox-run`
- `POST /proposed-changes/{id}:approve`
- `POST /proposed-changes/{id}:apply` (executes via apply_via_hook_id)

The v1 endpoints (assets, obligations, events, policy, sandbox-runs, evidence, audit-packets) are preserved.

3.4 The instrumentation SDK contract

Unchanged for supervision use. The same SDK is used internally by the Workflow Runtime to instrument every workflow agent step, which is how workflow agents become first-class governed assets without any special-case code.

3.5 The hook contract (MCP-first)

A hook is registered with the platform via `POST /hooks`. For an MCP-backed hook the registration body is:

```
json
{
  "name": "github_mcp",
  "kind": "mcp",
  "direction": "both",
  "endpoint": "https://mcp.github.com/sse",
  "auth_ref": "secret:github_pat",
  "scopes": ["repo:read", "repo:pr:write"],
  "effect_radius": "external_trusted"
}
```

At runtime, when a step says `type: hook.in, hook: github_mcp`, the Hook Layer opens an MCP session, invokes the relevant resources or tools, captures the event, and returns the payload to the step. Outbound `hook.out` works symmetrically; an outbound call's `effect_radius` determines whether a `gate.human` is required upstream.

For native connectors the `kind` is `native` and the platform ships an in-process adapter. For arbitrary HTTP, `kind` is `http` and the platform proxies via a configurable schema.

Part 4: Workflow Runtime, Sandbox, and Observability

This part covers the deepest differentiators in detail.

4.1 Workflow Runtime execution model

A WorkflowRun is a state machine over its DAG. The runtime maintains:

- The DAG (frozen at run start; later edits to the workflow definition do not affect in-flight runs).
- The set of completed step runs and their outputs (the "world state" available to downstream steps via templating).
- The current frontier (steps whose dependencies are satisfied and which are ready to dispatch).
- The hash chain for the run.

Step dispatch order is topological. Branching happens via `gate.policy` outcomes. Parallelism is supported when the DAG has independent branches; the runtime caps concurrent agent steps per run to avoid cost and resource overruns.

Each step type has a dedicated executor:

- `agent` executor builds a sandbox manifest (system prompt, tools, corpus access, inputs), launches a sandbox via the orchestrator, opens a WebSocket-based bridge to stream events back, and waits for structured output.
- `corpus.query` executor hits the corpus index directly (no sandbox needed; it's read-only).
- `hook.in` and `hook.out` executors call the Hook Layer.
- `transform` executes a small safe expression language (no arbitrary code in the demo; restricted to filtering, projection, and merging).
- `gate.policy` calls the policy engine with the step inputs as context.
- `gate.human` writes an Approval record and waits.
- `finding.emit` and `change.propose` write structured records to the database.

Each step run produces events on the bus and updates the WorkflowRun's progress in real time over the WebSocket stream to the UI.

Failure handling: a failed step can be configured to halt the run, branch to a fallback, or escalate to a human. By default, failures halt and emit a `workflow.run.finished` with `status="failed"`.

4.2 The agent step in detail (the moat)

The `agent` step is what makes Praetor different from Zapier. A step's manifest looks like:

json

```
{
  "step_id": "scan",
  "agent": {
    "model": "claude-sonnet-4",
    "system_prompt_ref": "prompts/code_compliance_scanner.md",
    "tools": ["grep", "ast_parse", "embed_search"],
    "memory": { "kind": "ephemeral_vector", "store_id": "scan-mem" },
    "corpora": ["urn:praetor:corpus:demo:gdpr",
                "urn:praetor:corpus:demo:iso_42001",
                "urn:praetor:corpus:demo:internal_data_min"]
  },
  "sandbox": { "mem_mb": 2048, "wall_s": 240, "network": "mocks_only" },
  "inputs": { "files": ["..."] },
  "expected_output_schema": { "findings": "list[Finding]" },
  "policy_set": "praetor.controls.workflow_agent_step"
}
```

When the step runs:

1. The orchestrator launches a sandbox with the praetor SDK preinstalled and configured for this step's `workflow_run_id` and `step_id`.
2. The agent's tools include both step-declared tools (`grep`, `ast_parse`, etc.) and platform-provided primitives (`corpus_query`, `cite_obligation`, `emit_finding`, `propose_change`).
3. Every tool call is governed by the policy engine. Calling `corpus_query` is unrestricted; calling `emit_finding` is allowed only if the output validates against the Finding schema; calling `propose_change` requires the proposed diff to fit within the step's declared `effect_scope`.
4. Memory writes are inspected for taint and classification.
5. The step's structured output is validated against `expected_output_schema` before being passed to downstream steps.

The agent runs unbounded (within the wall clock) and reasons over the corpus rather than executing a fixed sequence. This is the categorical difference from Zapier: a step is reasoning, not plumbing.

4.3 Self-governance: workflow agents as supervised assets

The single technical decision that makes the demo punch above its weight: workflow agents are first-class Assets of `type = "workflow_agent"`. The Inventory service has them, the Asset Detail UI works on them unchanged, the policy engine evaluates the same controls against their events, and the audit packet covers them.

What this means for buyers: the AI doing your compliance work is itself under your compliance watch. The same memory inspector, the same tool-permission policy, the same hash chain, the same audit packet. An auditor reviewing your audit packet sees both the work the agent did and the controls under which it was allowed to do it.

Concretely: when the workflow runtime launches an agent step, the agent's events are tagged with the `workflow_agent` asset URN. The supervision UI's live view works on workflow agents identically to production agents. A compliance officer can sit in the supervision view and watch the workflow agent reason in real time, complete with policy decisions on every tool call.

4.4 Sandbox design

Each sandbox is a short-lived Docker container.

Property	Value
Base image	<code>praetor/sandbox-runtime:latest</code> (Debian slim + Python + Node + the asset's deps)
Network	Custom bridge with only <code>praetor-mocks</code> and a controlled MCP-bridge container reachable.
FS isolation	Read-only root, writable overlay at <code>/sandbox/work</code> .
Secrets	None real. Mock credentials injected as env; outbound calls hit <code>praetor-mocks</code> .
Telemetry	Praetor SDK preinstalled and auto-configured.
Resource caps	2 vCPU, 2 GB RAM, 300 s wall clock (configurable per step).
Inputs	JSON manifest at <code>/sandbox/inputs/manifest.json</code> .
Outputs	<code>/sandbox/work/result.json</code> , validated against <code>expected_output_schema</code> .
Stronger isolation	<code>runsc</code> (gVisor) when available.

The MCP-bridge container is the new addition. It mediates outbound MCP calls from the sandbox to registered hooks, applying per-step scopes so that an agent step cannot reach beyond what its declaration allows. This is how workflow agents talk to GitHub, Slack, etc. through hooks without ever touching the public internet directly.

4.5 Memory inspector

Unchanged from v1 in mechanism. New in scope: applies to workflow agent memory just as it applies to production agent memory. When a workflow agent retrieves a chunk from a corpus, the chunk enters the agent's working memory tagged with provenance back to the document and citation. If a downstream tool call would surface that chunk's content into an external system, the tool permission policy can evaluate the chunk's taint and classification, not just the call's surface arguments.

4.6 Tool permission as policy

Unchanged. Now applies to:

- The supervised agent's tools (v1).
- The workflow agent's tools (new).
- The platform-provided primitives like `corpus_query`, `emit_finding`, `propose_change` (new). These have their own controls, e.g. `propose_change` may not produce a diff that touches files outside the step's declared `effect_scope`.

4.7 Approval workflow

Two channels at hackathon scope: in-product and Slack via MCP. Approvals are events. Both `gate.human` steps in workflows and remediation approvals in supervision use the same Approval record.

4.8 Audit packet

Now covers both surfaces. Structure:

1. Cover page (assets in scope, period, operator role, obligation set, packet hash).
2. Executive summary.
3. Obligation chain diagrams.
4. Workflow runs in the period (one section per run, with step timeline, findings, proposed changes, sandbox results).
5. Supervision evidence (controls tested, decisions, violations, applied remediations).
6. Hook activity ledger (inbound and outbound, with redactions).
7. Approval chain.
8. Evidence appendix with hash-chained samples.
9. Signing block.

The audit packet's value proposition is tangible: the customer's auditor sees not just *what* was done but *under what controls* and *with what evidence*, including for the AI-driven work itself.

Part 5: Implementation Roadmap

A 60-hour build divided into five phases.

5.1 Hour 0 to 6: Foundations

- Repo bootstrapped: `apps/web`, `apps/api`, `apps/sandbox`, `apps/workflow`, `packages/sdk-py`, `packages/sdk-ts`, `packages/hooks`, `infra/compose`.
- `docker compose` up: Postgres+pgvector+Timescale, Redis, MinIO, OPA, FastAPI, Next.js, plus three stub MCP servers (GitHub, Slack, local-files).
- Database schema migrated.
- Auth stub.
- CI smoke test.

Deliverable: empty dashboard at `localhost:3000` wired to a real API.

5.2 Hour 6 to 18: Two-surface vertical slice

The most important phase. Both surfaces get a minimal end-to-end path.

- **Supervision path:** SDK, telemetry ingest, one OPA policy on `northwind.send_email`, Asset Detail live view.
- **Workflow path:** corpus ingestion (a single markdown policy doc + a small ISO 42001 excerpt), one Workflow definition (a stripped-down code compliance scan), one MCP hook (a stub GitHub MCP that returns canned files), Workflow Runtime executes one `agent` step in a sandbox, emits one Finding.

Deliverable: trigger the demo agent against an injection prompt, watch the block live; separately, run the workflow, watch the agent step execute in a sandbox, see a Finding appear. Both views work.

5.3 Hour 18 to 36: Differentiators

- Workflow Runtime fleshed out: full DAG support, all step types, templating between steps, parallelism, gate.human and gate.policy.
- Hook Layer with at least three working MCP integrations (GitHub for repos and PRs, Slack for approvals, local-files for arbitrary documents).
- Corpus Management with ingestion of the five seed documents (EU AI Act excerpt, ISO 42001 excerpt, GDPR Article 5, internal data minimisation policy, OWASP Agent Top 10).
- Memory inspector wired for both workflow agents and production agents.
- Sandbox orchestrator with MCP-bridge container.
- Remediation as a workflow (so the same engine handles both flows).
- Two workflow templates wired end-to-end: Code Compliance Scan and Process Compliance Scan.
- UI: Workflows view, Workflow Run view, Hooks view, Corpora view.

Deliverable: end-to-end run of the unified demo flow described in Part 1.8, missing only audit packet polish.

5.4 Hour 36 to 50: Evidence and integration

- Evidence Generator subscribes to events and writes EvidenceRecords across both surfaces.
- Audit packet PDF generation covering workflow runs and supervision in one document.
- Self-governance UI polish: side-by-side panel showing the workflow agent's live trace alongside its policy feed and memory inspector, identical to the production agent supervision view.
- Multi-asset seed: three additional supervised agents and two pre-canned past workflow runs so the dashboard looks alive.
- Visual polish on the live three-pane view.

5.5 Hour 50 to 60: Demo lock

- Scripted demo flow rehearsed three times.
- Backup video.
- Pitch deck finalised with the Zapier-differentiation slide.
- `make demo` resets state and seeds.
- `README.md` covers setup, the demo command, and a one-paragraph elevator pitch.

5.6 Repository layout

```
praetor/
├─ apps/
│   └─ api/
│       └─ praetor_api/
│           ├── main.py
│           ├── routers/
│           │   ├── assets.py
│           │   ├── obligations.py
│           │   ├── events.py
│           │   ├── policy.py
│           │   ├── workflows.py
│           │   ├── workflow_runs.py
│           │   ├── hooks.py
│           │   ├── corpora.py
│           │   ├── findings.py
│           │   ├── proposed_changes.py
│           │   ├── sandbox.py
│           │   └─ evidence.py
│           ├── services/
│           │   ├── inventory.py
│           │   ├── obligation_mapper.py
│           │   ├── policy_hot.py
│           │   ├── policy_warm.py
│           │   ├── memory_inspector.py
│           │   ├── corpus_index.py
│           │   ├── evidence.py
│           │   └─ audit_packet.py
│           ├── models/
│           ├── ws/
│           └─ settings.py
│   └─ alembic/
├─ workflow/                                # the runtime
│   └─ praetor_workflow/
│       ├── runtime.py
│       ├── dag.py
│       ├── executors/
│       │   ├── agent_step.py
│       │   ├── corpus_query.py
│       │   ├── hook_in.py
│       │   ├── hook_out.py
│       │   ├── transform.py
│       │   ├── gate_policy.py
│       │   ├── gate_human.py
│       │   ├── finding_emit.py
│       │   └─ change_propose.py
│       └─ templating.py
```

```
└─ templates/
  └─ code_compliance_scan.yaml
  └─ process_compliance_scan.yaml
  └─ vendor_risk_assessment.yaml
  └─ policy_gap_analysis.yaml
  └─ continuous_control_monitoring.yaml
└─ sandbox/
  └─ praetor_sandbox/
    └─ orchestrator.py
    └─ docker_runtime.py
    └─ replay.py
    └─ mcp_bridge.py
    └─ harness/entrypoint.py
  └─ images/runtime/Dockerfile
└─ web/
  └─ app/
    └─ (dashboard)/page.tsx
    └─ inventory/page.tsx
    └─ assets/[id]/page.tsx
    └─ workflows/page.tsx
    └─ workflow-runs/[id]/page.tsx
    └─ corpora/page.tsx
    └─ hooks/page.tsx
    └─ obligations/page.tsx
    └─ sandbox/page.tsx
    └─ evidence/page.tsx
  └─ components/
    └─ agent-detail/
    └─ memory-inspector/
    └─ policy-feed/
    └─ workflow-graph/
    └─ workflow-run/
    └─ corpus-search/
    └─ hook-config/
    └─ finding-card/
    └─ proposed-change/
    └─ audit-packet/
└─ demo-agent/
  └─ northwind/
└─ packages/
  └─ sdk-py/
  └─ sdk-ts/
  └─ hooks/
    └─ mcp_client/
    └─ connectors/
    └─ http_proxy/
```

```

├─ content/
│   ├── obligations/
│   │   ├── eu_ai_act.yaml
│   │   ├── iso_42001.yaml
│   │   ├── nist_ai_rmf.yaml
│   │   ├── gdpr.yaml
│   │   ├── owasp_agent.yaml
│   │   └─ mitre_atlas.yaml
│   ├── controls/
│   │   ├── memory.rego
│   │   ├── tool_permission.rego
│   │   ├── output_filter.rego
│   │   ├── workflow_agent_step.rego
│   │   ├── workflow_findings_gate.rego
│   │   └─ hook_out_gate.rego
│   ├── corpora_seed/
│   │   ├── eu_ai_act_excerpt.md
│   │   ├── iso_42001_excerpt.md
│   │   ├── gdpr_article_5.md
│   │   ├── internal_data_min_policy.md
│   │   └─ owasp_agent_top_10.md
│   └─ prompts/
│       ├── code_compliance_scanner.md
│       ├── process_compliance_scanner.md
│       ├── vendor_risk_assessor.md
│       ├── policy_gap_analyst.md
│       └─ remediation_proposer.md
├─ infra/
│   ├── compose/
│   │   ├── docker-compose.yml
│   │   └─ .env.example
├─ scripts/
│   ├── seed_demo.py
│   └─ demo_run.sh
├─ docs/
│   ├── PRAETOR_PLAN.md
│   ├── ARCHITECTURE.md
│   ├── WORKFLOW_AUTHORING.md
│   └─ DEMO.md
├─ Makefile
└─ README.md

```

5.7 Key code patterns (illustrative)

5.7.1 Workflow Runtime executor dispatch

python

```
# apps/workflow/praetor_workflow/runtime.py
async def advance(run: WorkflowRun) -> WorkflowRun:
    frontier = run.dag.ready_steps(completed=run.completed_step_ids)
    for step in frontier:
        executor = EXECUTORS[step.type]
        await emit("workflow.step.started",
                  workflow_run_id=run.id, step_id=step.id, step_type=step.type)
    try:
        result = await executor.run(step, run.world_state())
        run.record_step_result(step.id, result)
        await emit("workflow.step.finished",
                  workflow_run_id=run.id, step_id=step.id,
                  status=result.status, outputs_redacted=redact(result.outputs))
    except StepFailed as e:
        await emit("workflow.step.finished",
                  workflow_run_id=run.id, step_id=step.id,
                  status="failed", errors=[str(e)])
        run.fail()
    return run
if run.is_complete():
    run.finish()
return run
```

5.7.2 Agent step executor

python

```
# apps/workflow/praetor_workflow/executors/agent_step.py
async def run(step, world):
    manifest = build_manifest(step, world) # prompts, tools, corpora, i
    sandbox = await orchestrator.launch(manifest, tag={
        "workflow_run_id": world.run_id, "step_id": step.id,
        "asset_urn": f"urn:praetor:asset:workflow_agent:{world.run_id}:{step.id}",
    })
    async for event in sandbox.stream(): # events flow back tagged
        await bus.publish(event)
    result = await sandbox.wait_for_result()
    validated = validate_against_schema(result, step.expected_output_schema)
    return StepResult(status="succeeded", outputs=validated)
```

5.7.3 MCP-backed hook call

python

```
# packages/hooks/mcp_client/client.py
async def call_in(hook: Hook, scope: str, args: dict) -> dict:
    async with mcp_session(hook.endpoint, hook.auth_ref) as session:
        if scope.endswith(":read"):
            return await session.list_resources(filter=args)
        if scope.endswith(":pr:write"):
            return await session.call_tool("create_pull_request", args)
        raise UnknownScope(scope)
```

5.7.4 Corpus query primitive (callable as a tool from any agent step)

python

```
# apps/api/praetor_api/services/corpus_index.py
async def hybrid_search(corpus_id, query, filters=None, k=8):
    embedding = await embed(query)
    dense = await db.execute(VECTOR_QUERY, embedding, corpus_id, k=k*2)
    keyword = await db.execute(TS_QUERY, query, corpus_id, k=k*2)
    merged = rrf_merge(dense, keyword, k=k)
    chunks = [hydrate(r) for r in merged]
    await emit("corpus.query",
               corpus_id=corpus_id, query=query,
               chunks_returned=[c.summary() for c in chunks],
               top_score=merged[0].score if merged else 0.0)
    return chunks
```

5.7.5 Finding emission (tool exposed to workflow agents)

python

```
# tool surface inside the sandbox
async def emit_finding(*, title: str, description: str,
                      severity: str, obligations_cited: list[str],
                      documents_cited: list[dict], confidence: float,
                      target_asset_id: str | None = None) -> str:
    finding = Finding(title=title, description=description, severity=severity,
                     obligations_cited=obligations_cited,
                     documents_cited=documents_cited,
                     confidence=confidence,
                     asset_id=target_asset_id)
    finding_id = await transport.create_finding(finding)
    await emit("finding.emitted", finding_id=finding_id,
              severity=severity, obligations_cited=obligations_cited)
    return finding_id
```

5.8 Demo data

`scripts/seed_demo.py` provisions:

- One AI system (`Northwind Patient Support`) and one production agent (`support-bot`) with three tools.
- Five corpora (the seed documents listed in 5.6).
- Five workflow templates ready to instantiate.
- Three pre-configured hooks (stub GitHub MCP serving canned files, stub Slack MCP for approvals, local-files MCP for documents).
- Two pre-canned past workflow runs so the dashboard is populated.
- A canned violation scenario for the supervision demo.

`make demo` resets state and seeds.

Part 6: Demo Script and Judging Strategy

6.1 The five-minute demo

00:00 to 00:30 — Frame the problem. "Compliance teams have two new problems: they spend half their week doing repetitive analysis (mapping a new regulation to existing controls, scanning code for policy gaps, assessing vendors) and they're now also being asked to govern AI agents in production. Praetor solves both with one control plane. The agents we run to do your compliance work are themselves governed by the same runtime we offer for your production AI."

00:30 to 01:30 — Trigger the workflow. Open the Workflows view. Click *Code Compliance Scan*. Configure inputs: GitHub repo URL (Northwind support-bot), corpora to reason over (internal data minimisation, ISO 42001, GDPR, OWASP Agent Top 10). Run.

01:30 to 02:30 — Watch the workflow execute. The Workflow Run view shows the DAG progressing. Click the running `scan` step. The Asset Detail panel slides in: the agent's thoughts on the left, memory writes in the centre with citations back to the corpora, policy decisions on the right. Note out loud: this is the *workflow agent*. It is being governed by the same runtime that governs the customer's production AI. Memory writes have provenance back to the regulation chunks. Tool calls are policy-checked.

02:30 to 03:15 — Finding and proposed change. The scan finishes. A Finding appears: the support-bot's `send_email` tool does not validate recipient domains against the data minimisation policy. Citations to internal policy section 3.2, ISO 42001 8.3, GDPR 5(1)(c). Confidence 0.92. Click into it: a ProposedChange is attached, a code patch tightening the validator. The patch ran in a sandbox against a battery of replayed inputs, including known prompt injection patterns, all green.

03:15 to 03:45 — Approve and apply. Approve in-product. The change applies via the GitHub MCP hook as a pull request. The PR appears in the Hooks view's recent calls.

03:45 to 04:30 — Switch to Supervision. Open the Northwind support-bot in the Inventory. Replay the malicious-email injection prompt. The agent's `send_email` tool refuses internally because the validator catches it. The policy engine logs a refusal as evidence rather than as a violation. Show the live three-pane view: identical layout to the workflow agent we watched earlier. Same product, two surfaces.

04:30 to 05:00 — Audit packet. Click Generate Audit Packet. PDF appears covering: the workflow run, the finding, the proposed change, the approval chain, the applied PR, the supervision evidence afterwards, the obligation graph, and the signed evidence samples. Open it, scroll once, close. End on the title slide: *The governed runtime for agentic GRC*.

6.2 Anticipated judge questions

- *"Isn't this just Zapier with extra steps?"* → No. A Zapier step is a SaaS API call. A Praetor step is a sandboxed agent reasoning over a regulatory corpus, governed by the same control plane that watches our customers' production AI. Different primitive, different audit posture, different output type (structured Finding versus side effect).
- *"How do you compete with Vanta or Drata?"* → We don't displace them, we integrate. They are great at evidence collection. We are the layer above: agents that do the analytical work, plus runtime governance for AI. We pull asset inventory from them on day one.
- *"What stops the workflow agent from making things up?"* → Three things. Outputs are validated against typed schemas. Citations are constrained to the ingested corpus. Confidence below threshold routes to human review. And the workflow agent's reasoning is itself governed by the same policy engine, including memory provenance and tool permission.
- *"Latency?"* → Hot-path policy decisions are sub-10ms (OPA). Workflow runs are not on a request critical path. Supervision warm-path is async.
- *"Self-governance feels like marketing."* → It's literal. Workflow agents are Assets of a specific type. The Asset Detail view works identically. The audit packet covers them. The hash chain is per-asset and includes them. Show me a competitor where the AI doing the GRC work appears in the same UI as the AI being governed.
- *"What's the moat?"* → Domain primitives, the corpus and obligation library, MCP-native hook surface, sandboxed step execution as default, and self-governance. The first two compound with usage. The last three are architectural and hard to retrofit.

6.3 Backup plan

Pre-recorded screen capture, `make demo` for instant resets, a "sandbox replay" mode that plays a captured run from disk if live LLM latency spikes.

Part 7: Post-Hackathon Roadmap

7.1 Next 30 days

- Replace dev auth with multi-tenant auth (WorkOS or Auth0).
- Sandbox runtime to Firecracker with a warm pool of microVMs.
- Workflow template marketplace skeleton (community submissions).
- MCP integrations: Linear, Notion, Confluence, Drive, Jira (full-fat).
- Vanta and Drata native connectors for inventory and evidence sync.
- Slack approvals via official MCP.

7.2 Next 90 days

- TypeScript and Go SDKs.
- Sidecar for OpenAI Assistants and Microsoft Copilot Studio.
- Counterfactual replay productised as "policy preview."
- SOC 2 Type I and ISO 42001 readiness packet templates as out-of-the-box workflows.
- Workflow scheduling with retry semantics and partial replay.
- Public API rate limiting, usage metering, billing skeleton.

7.3 Open questions

- Which jurisdiction packs to ship first beyond EU and US baseline. Australia (Privacy Act reforms, Voluntary AI Safety Standard) is a natural early addition; UK and Singapore close behind.
- Whether to invest in our own obligation content team or partner with an existing content vendor.
- Whether to expose the sandbox runtime as its own product (a "secure agent execution" SKU).
- How aggressively to chase the workflow template marketplace; could be a defensible distribution channel or a distraction.

Appendix A: Risks and mitigations

Risk	Mitigation
Scope blowout in 60 hours	Two-surface vertical slice at hour 18 forces both modes to exist early. Cut depth on warm-path policy and memory poisoning heuristics first.
LLM latency in the demo	Hot path is OPA. Pre-warm Anthropic connections. Fallback to a smaller local model for <code>thought</code> events only.
Sandbox launch slow on first run	Pre-pull the runtime image at startup; warm pool of one container ready.
Obligation content errors	Confidence scoring + human review for low-confidence; explicit disclaimer that this is not legal advice.
Memory classifier false positives at demo time	Demo data curated. Threshold tuned for the demo asset.
MCP integrations flaky	Stub MCP servers shipped with the demo serve canned data; live MCP usage demoed only if stable.
Anthropic API key exhaustion	Two keys in rotation; local fallback model for low-stakes calls.
Live audience prompt injection	Demo runs against a captive playground; audience cannot directly inject. Q&A only after.
"Zapier already does this" objection	Differentiation slide and the live self-governance moment in the demo. Practised answer in 6.2.

Appendix B: Sample workflow template (full)

yaml

```
# apps/workflow/templates/code_compliance_scan.yaml
name: code_compliance_scan
description: >
  Scan a code repository against a set of compliance corpora.
  Emit findings with citations and propose remediations as code patches.
trigger: manual
inputs:
  repo_url: { type: string, required: true }
  branch:   { type: string, default: "main" }
  corpus_ids: { type: list[string], required: true }
  paths:      { type: list[string], default: ["**/*.py", "**/*.ts", "**/*.tsx"] }

required_hooks:
- github_mcp
required_corpora:
- any_of: "{{ inputs.corpus_ids }}"

steps:

- id: pull
  type: hook.in
  hook: github_mcp
  args:
    url:      "{{ inputs.repo_url }}"
    branch:   "{{ inputs.branch }}"
    paths:    "{{ inputs.paths }}"
  outputs: { files: list[File] }

- id: scan
  type: agent
  sandbox: { mem_mb: 2048, wall_s: 240, network: mocks_only }
  agent:
    model: claude-sonnet-4
    system_prompt_ref: prompts/code_compliance_scanner.md
    tools: [grep, ast_parse, embed_search, corpus_query, cite_obligation, emit_findings]
    memory: { kind: ephemeral_vector, store_id: "scan-{{ run.id }}" }
    corpora: "{{ inputs.corpus_ids }}"
  inputs: { files: "{{ steps.pull.files }}" }
  expected_output_schema:
    findings: list[Finding]

- id: gate
  type: gate.policy
  policy: praetor.controls.workflow_findings_gate
  inputs: { findings: "{{ steps.scan.findings }}" }
```

```

inputs: { findings: "{{ steps.scan.findings }}" }

- id: emit
  type: finding.emit
  inputs: { findings: "{{ steps.scan.findings }}" }

- id: propose
  type: agent
  sandbox: { mem_mb: 2048, wall_s: 300, network: mocks_only }
  agent:
    model: claude-opus-4
    system_prompt_ref: prompts/remediation_proposer.md
    tools: [read_repo, write_diff, run_tests, propose_change]
  inputs:
    findings: "{{ steps.scan.findings }}"
    repo:      "{{ steps.pull.files }}"
  expected_output_schema:
    proposed_changes: list[ProposedChange]

- id: approve
  type: gate.human
  routes_to: { role: "compliance_lead" }
  inputs: { changes: "{{ steps.propose.proposed_changes }}" }

- id: open_pr
  type: hook.out
  hook: github_mcp
  when: "{{ steps.approve.outcome == 'approved' }}"
  args:
    repo: "{{ inputs.repo_url }}"
    base: "{{ inputs.branch }}"
    head: "praetor/{{ run.id }}"
    title: "Praetor: compliance remediations"
    changes: "{{ steps.propose.proposed_changes }}"

```

Appendix C: Sample obligation content (excerpt)

yaml

```
# content/obligations/eu_ai_act.yaml
- id: urn:praetor:obligation:eu_ai_act:annex_iii_health
  framework: eu_ai_act
  citation: "Annex III(5)(a)"
  text: >
    AI systems intended to evaluate eligibility of natural persons for essential
    health services are classified as high-risk.
  applicability:
    sectors: ["health"]
    operator_roles: ["provider", "deployer"]
    jurisdictions: ["eu"]
    asset_types: ["ai_system", "agent"]
  severity_default: "block"
  version: "2026.04"
- id: urn:praetor:obligation:eu_ai_act:art_14_human_oversight
  framework: eu_ai_act
  citation: "Art 14(4)(c)"
  text: >
    High-risk AI systems shall be designed so that natural persons assigned to
    human oversight can correctly interpret the system's output.
  applicability:
    high_risk: true
  severity_default: "warn"
  version: "2026.04"
```

Appendix D: Sample control (excerpt)

```
rego
```

```
# content/controls/workflow_findings_gate.rego
```

```
package praetor.controls.workflow_findings_gate
```

```
default decision := {"outcome": "allow"}
```

```
# Block emission of any finding whose confidence is below threshold and which
```

```
# cites a high-severity obligation, unless reviewed by a human.
```

```
decision := {"outcome": "block", "rationale": rationale} {
```

```
    some f in input.findings
```

```
    f.confidence < 0.6
```

```
    some o in f.obligations_cited
```

```
    obligation := input.context.obligations[o]
```

```
    obligation.severity_default == "block"
```

```
    not f.reviewer
```

```
    rationale := sprintf(
```

```
        "Finding %v cites blocking obligation %v with confidence %.2f and lacks revi
```

```
        [f.id, o, f.confidence])
```

```
}
```

Appendix E: Glossary

- **Asset:** a governed entity in Praetor's inventory (production AI, workflow agent, workflow run, tool, memory store, dataset, model).
- **Corpus:** a versioned, queryable body of documents (regulations, standards, internal policies, code repos, process artefacts).
- **Obligation:** a rule from a regulatory framework that may apply to an asset.
- **Control:** a deterministic check that implements one or more obligations.
- **Workflow:** a typed DAG of steps with declared inputs, outputs, hooks, and policies.
- **WorkflowRun:** one execution of a Workflow; itself an Asset.
- **Hook:** an inbound or outbound bridge to an external system. MCP-backed by default.
- **Finding:** a structured output of a workflow agent step describing a discovered issue, with citations.
- **ProposedChange:** a structured remediation tied to a Finding, sandbox-tested before approval.
- **Decision:** the outcome of evaluating a control over an event.
- **Evidence record:** a signed artefact linking events, decisions, controls, and obligations.
- **Audit packet:** a bundle of evidence over a period, covering both surfaces.
- **Hot path:** sub-10ms deterministic policy evaluation.
- **Warm path:** asynchronous LLM-driven reasoning, never blocking.
- **Sandbox:** an isolated container where workflow agent steps and remediations run.
- **Quarantine:** a memory state where a write is stored but unreadable until reviewed.
- **Operator role:** the EU AI Act role (provider, deployer, distributor, importer) the customer's organisation plays for an asset.